



Digital Design Verification

Lab Manual # 26 – Cache Simulator Design

Release: 1.0

Date: 4-July-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	4/07/2024	Ali Aqdas	Complete manual	-



Contents

Objective.....	3
Tools and Technologies.....	3
Submission Guidelines.....	3
Downloading Support Files	3
Programming Task	5
Implementation Steps	7
Self Evaluation	7
Reference	8



Objective

This lab will help you understand how cache memories work. For this purpose, you will write a small C program that simulates the behavior of a cache memory. This will make you an expert in basics of cache systems and strengthen your C programming skills.

Tools and Technologies

- C/C++
- GNU Toolchain

Submission Guidelines

Write a short report documenting the functionality and features of the designed cache simulator and attach the source code alongside the document.

Downloading Support Files

Download the `cachelab-handout.zip` from LMS

Start by copying `cachelab-handout.zip` to a protected Linux directory in which you plan to do your work. Then give the command below

```
$ unzip cachelab-handout.zip
```

This will create a directory called `cachelab-handout` that contains several files.



The **traces** subdirectory of the handout directory contains a collection of reference trace files that we will use to evaluate the correctness of your cache simulator. The trace files are generated by a Linux program called **valgrind**. For example, typing

```
$ valgrind --log-fd=1 --tool=lackey -v --trace-mem=yes ls -l
```

on the command line runs the executable program, “**ls -l**” captures a trace of each of its memory accesses in the order they occur, and prints them on **stdout**.

Valgrind memory traces have the following form:

```
I 0400d7d4,8
M 0421c7f0,4
L 04f6b868,8
S 7ff0005c8,8
```

Each line denotes one or two memory accesses. The format of each line is

[space]operation address,size

The **operation** field denotes the type of memory access: “**I**” denotes an instruction load, “**L**” a data load, “**S**” a data store, and “**M**” a data modify (i.e., a data load followed by a data store). There is **never a space before each “I”**. There is **always a space before each “M”, “L”, and “S”**.

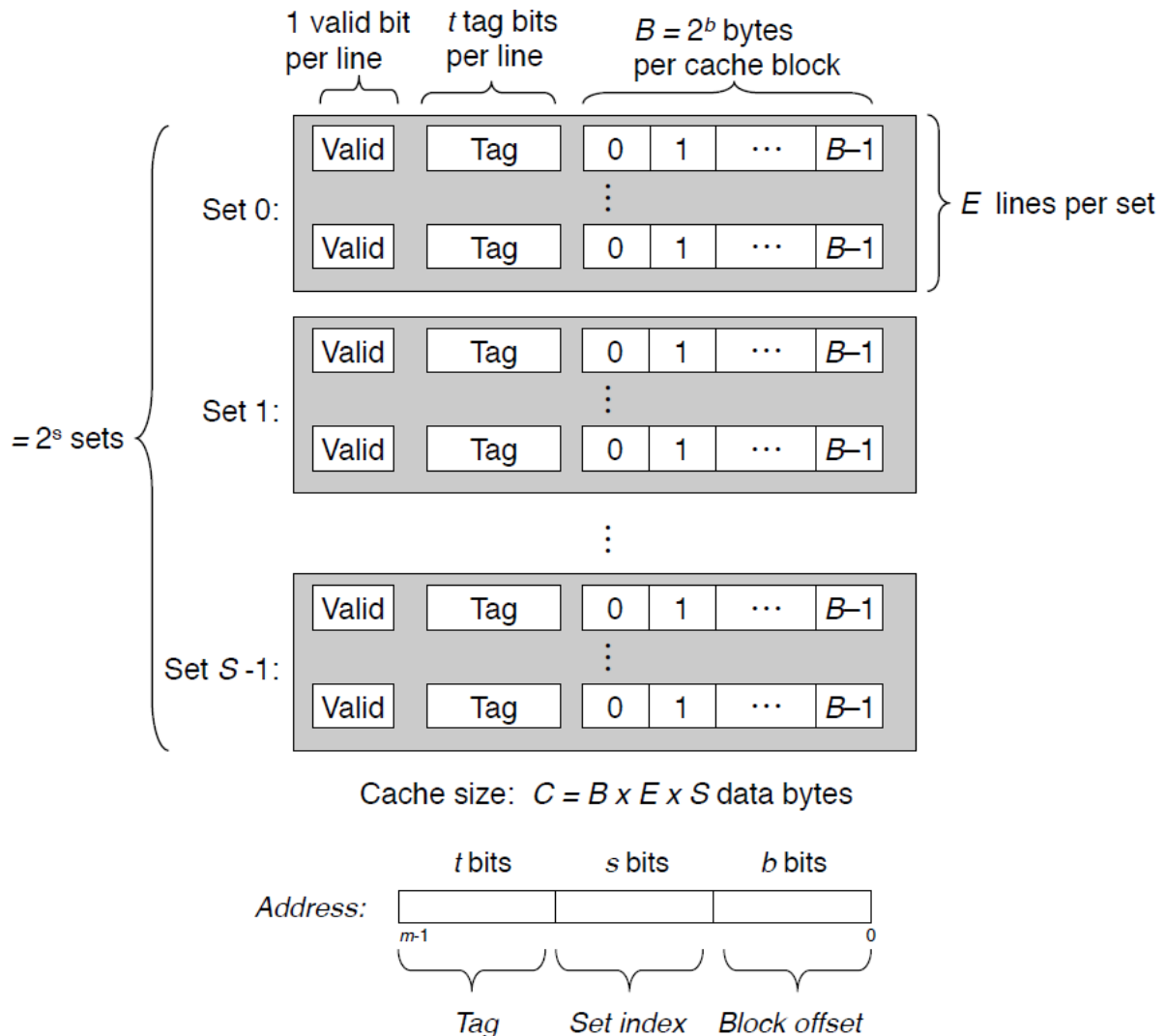
The **address** field specifies a 64-bit hexadecimal memory address.

The **size** field specifies the number of bytes accessed by the operation.

NOTE: You will NOT have to generate any traces on your own. You need to work only with the traces provided to you.



Figure 1: General organization of cache (S, E, B, m). (a) A cache is an array of sets. Each set contains one or more lines. Each line contains a valid bit, some tag bits, and a block of data, (b) The cache organization induces a partition of the m address bits into t tag bits, s set index bits, and b block offset bits.



Programming Task

You will write a cache simulator in the provided file `csim.c` that takes a `valgrind` memory trace as input, simulates the hit/miss behavior of a cache memory on this trace, and **outputs the total number of hits, misses, and evictions**.

We have provided you with the binary executable of a **reference cache simulator**, called `csim-ref`, that simulates the behavior of a cache with arbitrary size and associativity on a `valgrind` trace file. It uses the **LRU (least-recently used)** replacement policy when choosing which cache line to evict. The cache is organized so that it can find the requested word by simply inspecting the bits of the address, like a hash table with an extremely simple hash function.

The simulation follows the cache organization and notation (s , E , and b) as described in Computer Systems: A Programmer's Perspective, 3/E, Bryan, O'Hallaron textbook (see Figure 1 taken from the textbook [Page 616]). The s set index bits in the address form an



index into the array of s sets. The first set is set 0, the second set is set 1, and so on. When interpreted as an unsigned integer, the set index bits tell us which set the word must be stored in.

Once we know which set the word must be contained in, the t tag bits in the address tell us which line (if any) in the set contains the word. **A line in the set contains the word if and only if the valid bit is set and the tag bits in the line match the tag bits in the address.**

Once we have located the line identified by the tag in the set identified by the set index, then the b block offset bits give us the offset of the word in the B-byte data block.

The cache to be simulated uses **the LRU (least-recently used) policy** that works by replacing the least recently used lines first. **Your implementation needs to keep track of what cache line was used (and when!).** To that end, you can keep a counter for each cache line and update such counters accordingly. To understand how the LRU policy works, take a look at the example depicted in Figure 2, showing the access sequence of blocks as follows: **A B C D E D F**. In the example, blocks **A B C D** gets placed/accessed in the cache with the corresponding counters (i.e., increment 1 for each new access). Next, when **E** is accessed, it will result in a cache miss and there is a need for block replacement. According to the LRU Algorithm, since **A** has the lowest counter **A(0)**, then **E** will replace **A**.

The reference simulator takes the following command-line arguments:

```
Usage: ./csim-ref [-hv] -s <s> -E <E> -b <b> -t <tracefile>
```

- **-h**: Optional help flag that prints usage info
- **-v**: Optional verbose flag that displays trace info
- **-s <s>**: Number of set index bits ($S = 2^s$ is the number of sets)
- **-E <E>**: Associativity (number of lines per set)
- **-b **: Number of block bits ($B = 2^b$ is the block size)
- **-t <tracefile>**: Name of the **valgrind** trace to replay

For example:

```
$ ./csim-ref -s 4 -E 1 -b 4 -t traces/yi.trace  
hits:4 misses:5 evictions:3
```

The same example in verbose mode:

```
$ ./csim-ref -v -s 4 -E 1 -b 4 -t traces/yi.trace  
L 10,1 miss  
M 20,1 miss hit  
L 22,1 hit  
S 18,1 hit  
L 110,1 miss eviction  
L 210,1 miss eviction  
M 12,1 miss eviction hit  
hits:4 misses:5 evictions:3
```

We have provided you with skeleton code in file **csim.c**. Your job is to complete the implementation so that it outputs the correct number of hits, misses and evictions. You **NEED NOT** support the verbose output (using the **-v** option) as mentioned above.



Implementation Steps

Please review the following important points regarding your implementation. You should carefully read the skeleton code in `csim.c`. Also, you should read the comments in the file which provide additional details.

Starting from `main()` function, the flow is described below in brief:

- Parse the command line arguments. This is already done for you
- `void initCache()` - This function should allocate the data structures to hold information about the sets and cache lines using `malloc` depending on the values of parameters `S` ($S = 2^s$) and `E`. **You need to complete this function**
- `void replayTrace(char* trace_fn)` This function parses the input trace file. This part is already done for you. It should call the `accessData()` function. **You need to complete the missing code in this function.**
- `void accessData(mem_addr_t addr)` This function is the core of implementation which should use the data structures that were allocated in `initCache()` function to model the cache hits, misses and evictions. You need to complete this function. The most crucial thing is to update the global variables `hit_count`, `miss_count`, `eviction_count` inside this function appropriately. **You should implement Least-Recently-Used (LRU) cache replacement policy.**
- `void freeCache()` This function should free up any memory you allocated using `malloc()` in `initCache()` function. This is crucial to avoid memory leaks in the code. You need to complete this function.
- `printSummary(hit_count, miss_count, eviction_count)` This function prints the statistics in the desired format. **This is already implemented for you.**

In your implementation, you must adhere to the following rules

- Your `csim.c` file must compile **without warnings in order to receive credit**.
- Your simulator must work correctly for arbitrary `s`, `E`, and `b`. This means that you will need to dynamically allocate storage for your simulator's data structures using the `malloc` function.
- For this lab, we are interested only in data cache performance, so your simulator **should ignore all instruction cache accesses** (lines starting with "I"). Recall that `valgrind` always puts "I" in the first column (with no preceding space), and "M", "L", and "S" in the second column (with a preceding space). This may help you parse the trace.
- To receive credit, you must call the function `printSummary`, with the total number of hits, misses, and evictions, at the end of your `main` function.
- For this lab, you should assume that memory accesses are aligned properly, such that a single memory access never crosses block boundaries. By making this assumption, you can ignore the request sizes in the `valgrind` traces.

Important: Each data load (L) or store (S) operation can cause at most one cache miss. The data modify operation (M) is treated as a load followed by a store to the same address. Thus, an M operation can result in two cache hits, or a miss and a hit plus a possible eviction (even when only 1 byte is modified).

Self Evaluation

You can use the reference simulator `csim-ref` to obtain the correct answer for each of these test cases. During debugging, use the `-v` option for a detailed record of each hit and



miss.

Reference

1. Lab 05 Caches, CS449 Fall 2021, University of Pittsburgh